



1495
UNIVERSITY OF
ABERDEEN

CELEBRATING
525 YEARS
1495 – 2020

ABERDEEN 2040

Decision Trees

Data Mining & Visualisation
Lecture 11

2025

Today...

- Recap on trees
- Entropy
- Concept of 'purity of leaf node'
- Information Gain

Trees

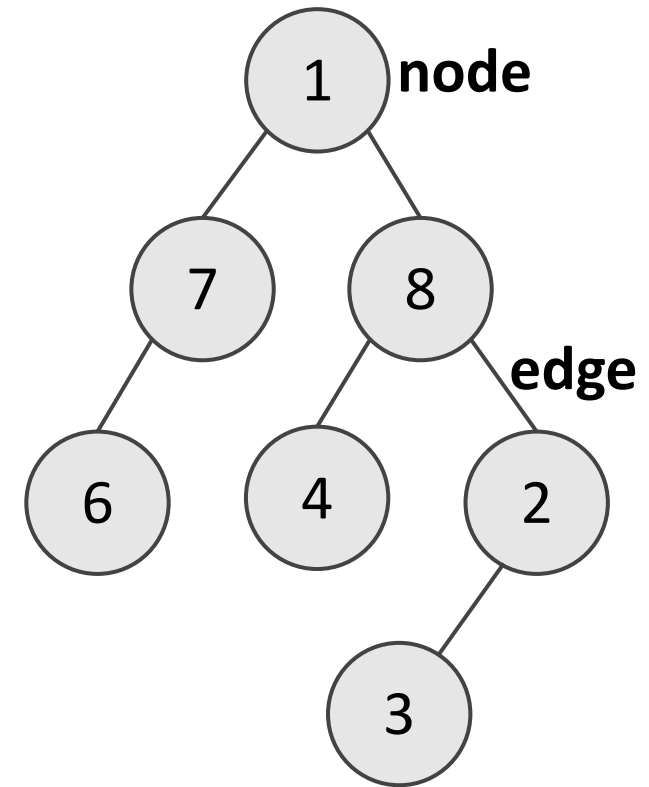
ABERDEEN 2040



Trees

A tree is a non-linear data structure that consists of nodes connected by edges.

They are commonly used across various topics in computer science.



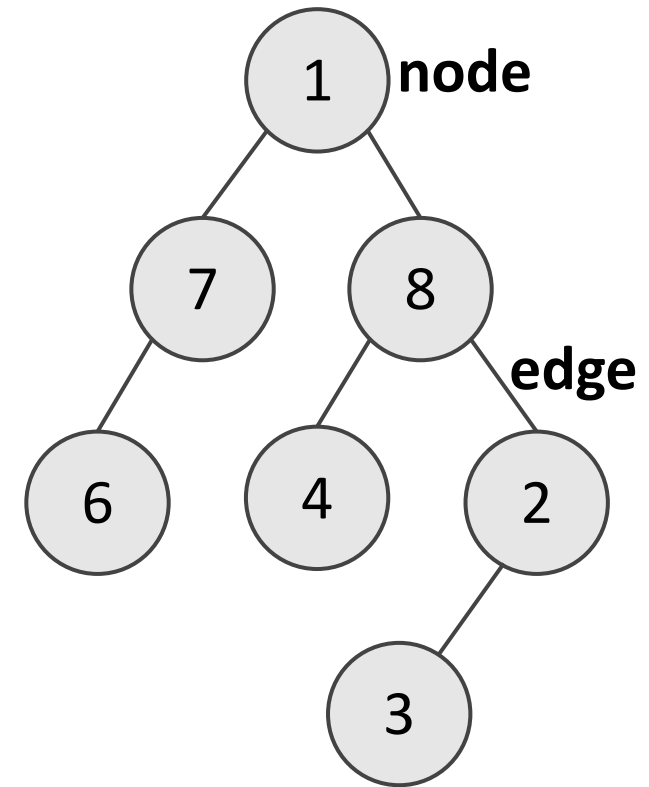
Trees

Trees are collections of *nodes*:

- They must have one root node, maximum!
- Each node has:
 - a value (e.g., a number), and
 - a list of references to other nodes
- A tree can be **empty**.

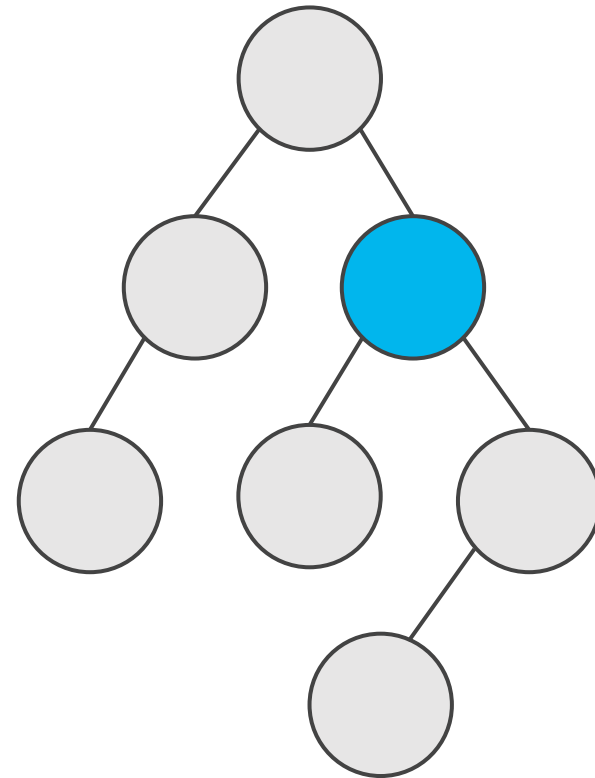
Edges represent connections between two nodes:

- They are sometimes called “arc”, “branch” or “link”.



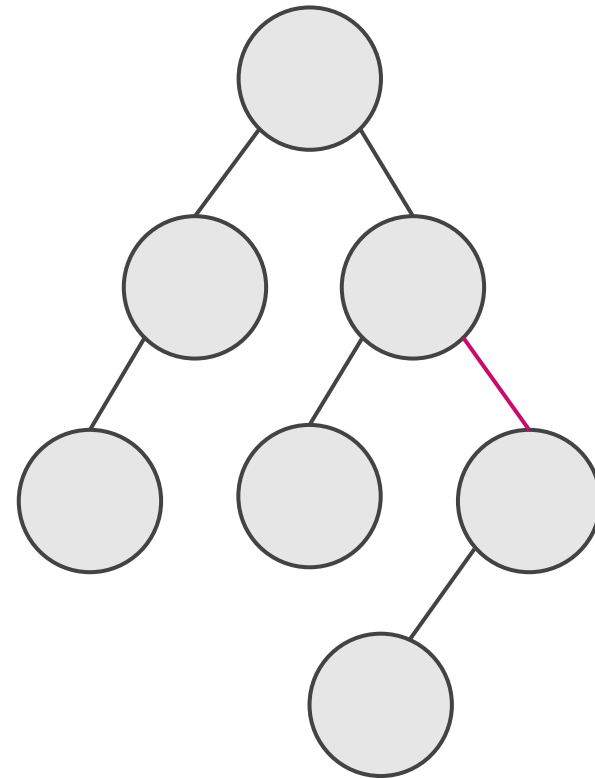
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



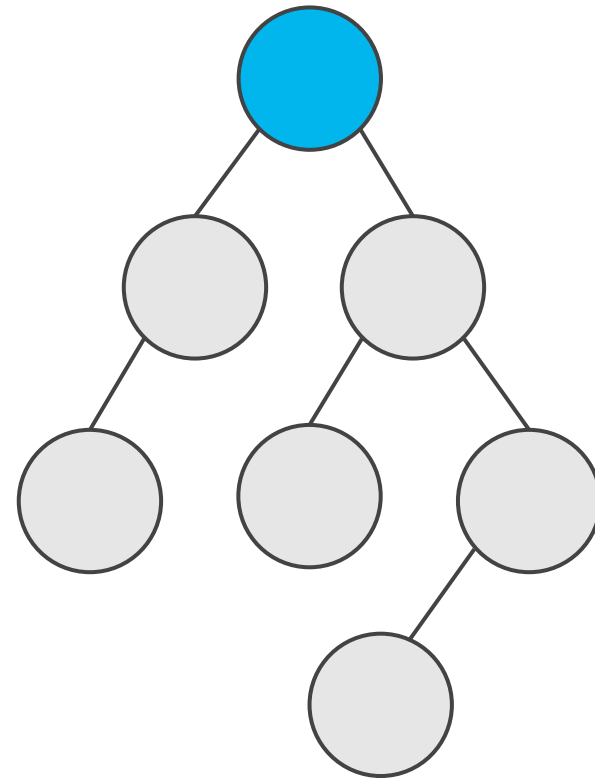
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



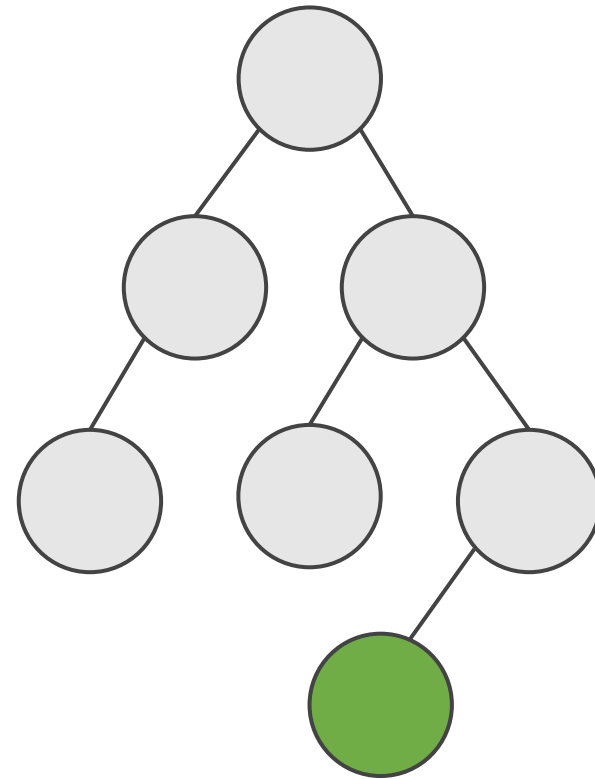
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



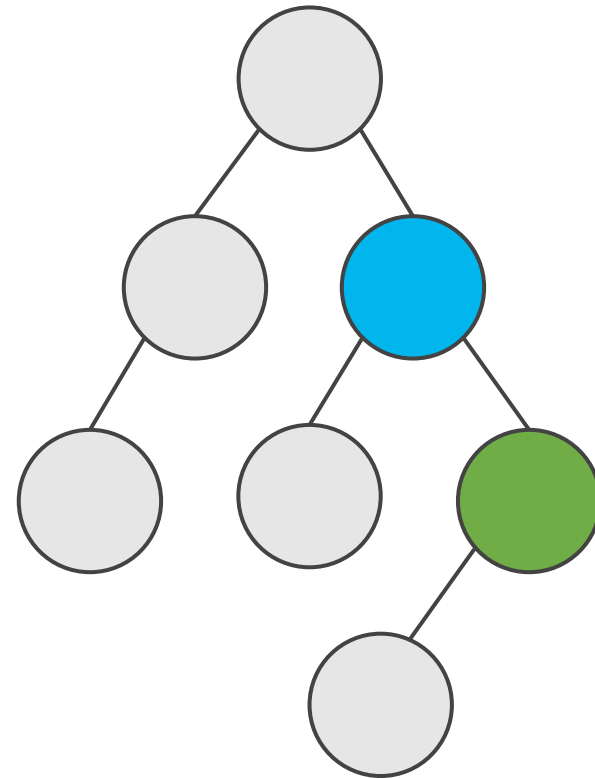
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



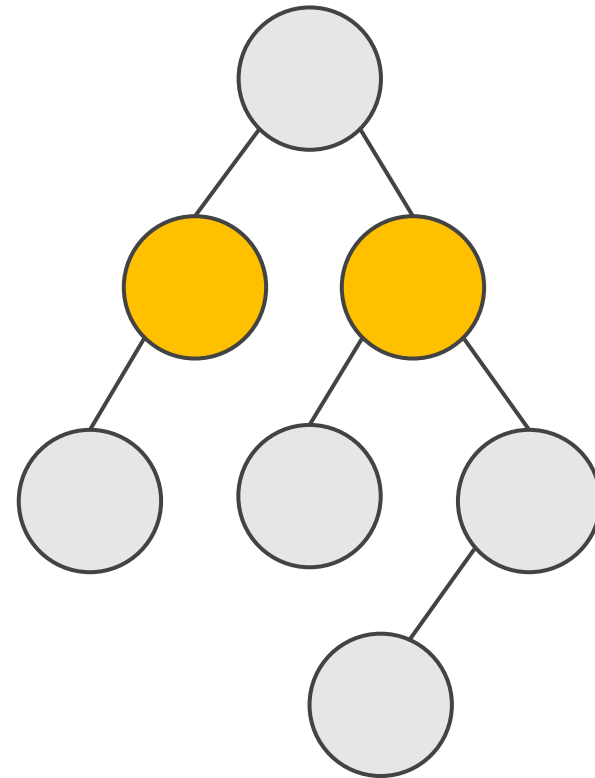
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



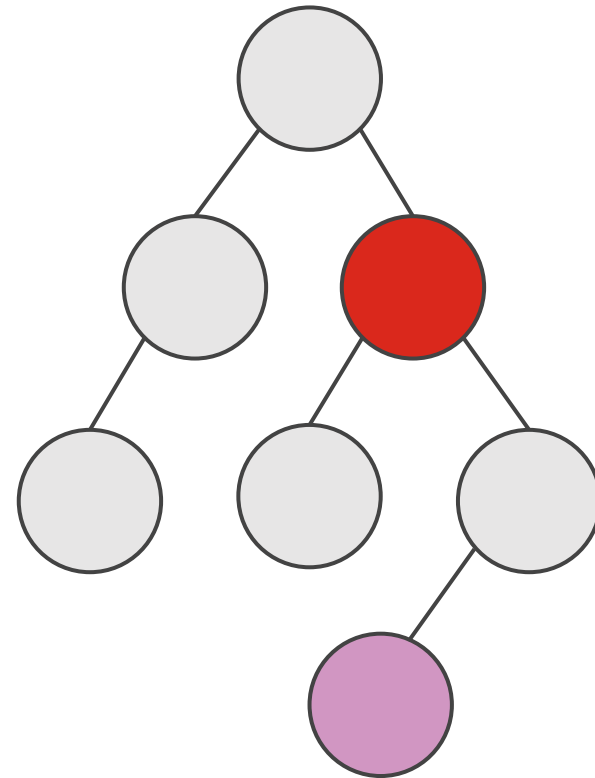
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



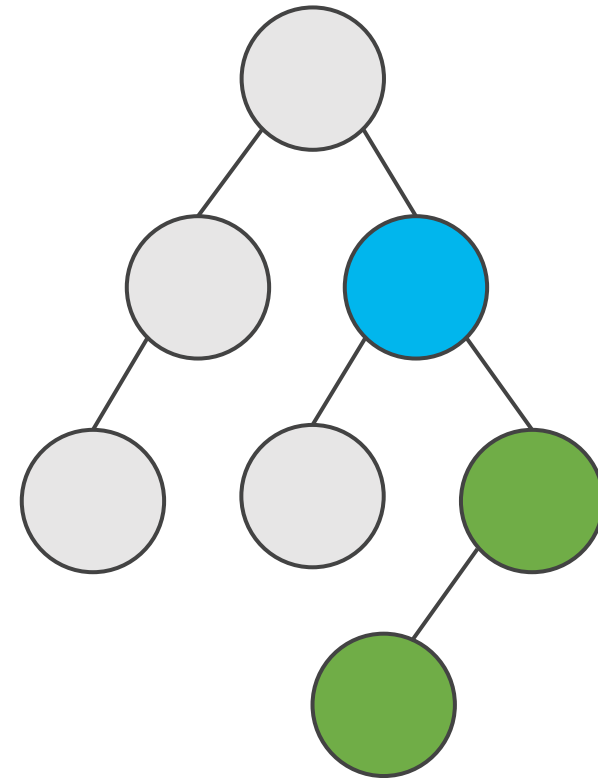
Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree



Tree Terminology

Node
Edge
Root
Leaf
Parent
Child
Siblings
Ancestor
Descendant
Subtree

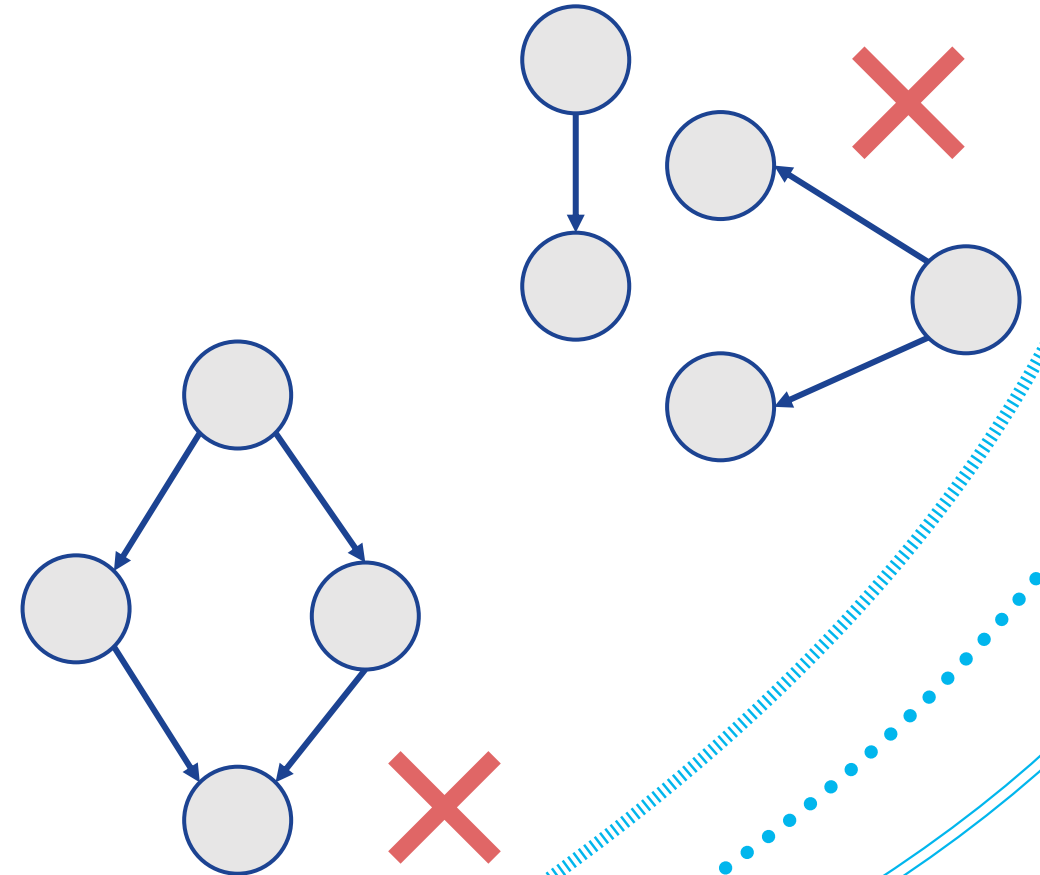
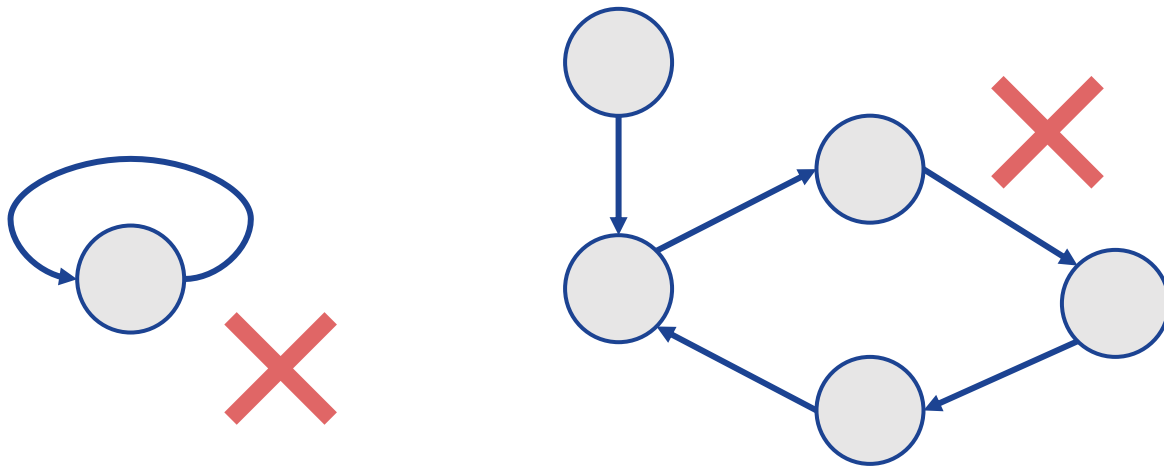


Tree Terminology

Root	The “top” node, and the only one without parents
Leaf	A node with no children
Parent	The node directly above a given node n in the tree
Child	The node directly below a given node n in the tree
Siblings	Nodes with the same parent node
Ancestor	An ancestor of n is a node along the path from the root to n
Descendant	A descendant of n is a node along the path from n to a leaf node
Subtree	A subtree of a node n is a tree rooted by a child node of n

Some Non-Trees

- The root cannot be a child node
- Trees can only have one root node
- No node can have more than one parent



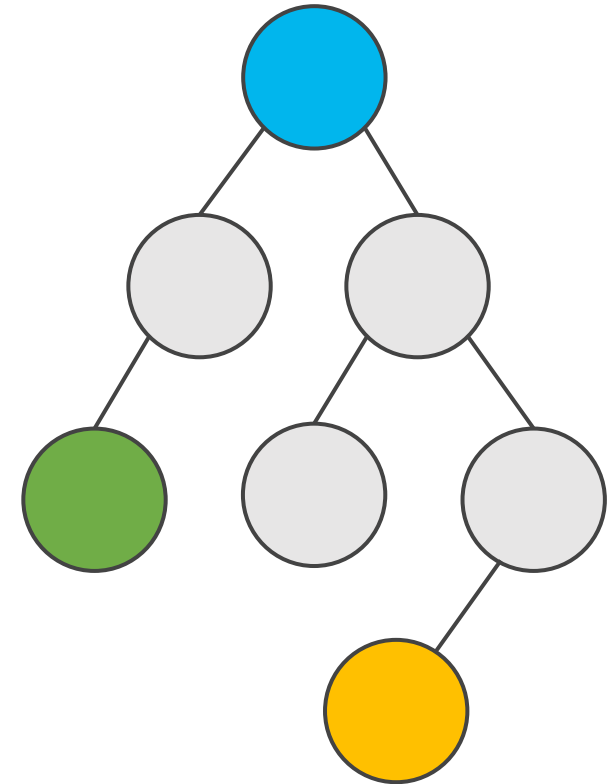
Path and Depth

A path is a sequence of connected edges.

For every node, there is a unique path to the root node (a definition of a tree!).

The depth of a node n is the length of the path (number edges).

What are the depths of the three highlighted nodes on the right?



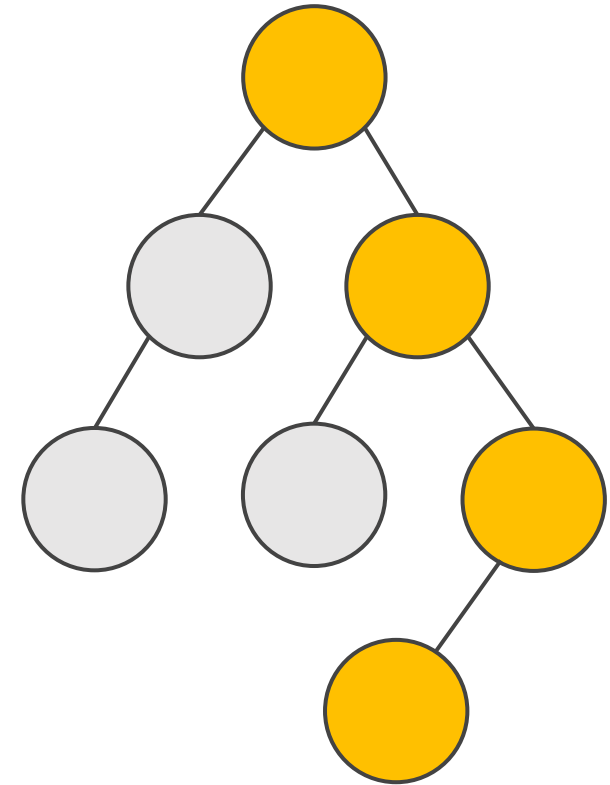
Height and Size

Height: is the longest path from root to leaf.

Size: the number of nodes in the tree.

Looking at the tree shown on the right:

- What is its height? 3
- What is its size? 7



Height and Size

Height: is the longest path from root to leaf.

Size: the number of nodes in the tree.

What about an empty tree?

- It's size is easy - zero (no nodes)
- ... but what about its height? -1

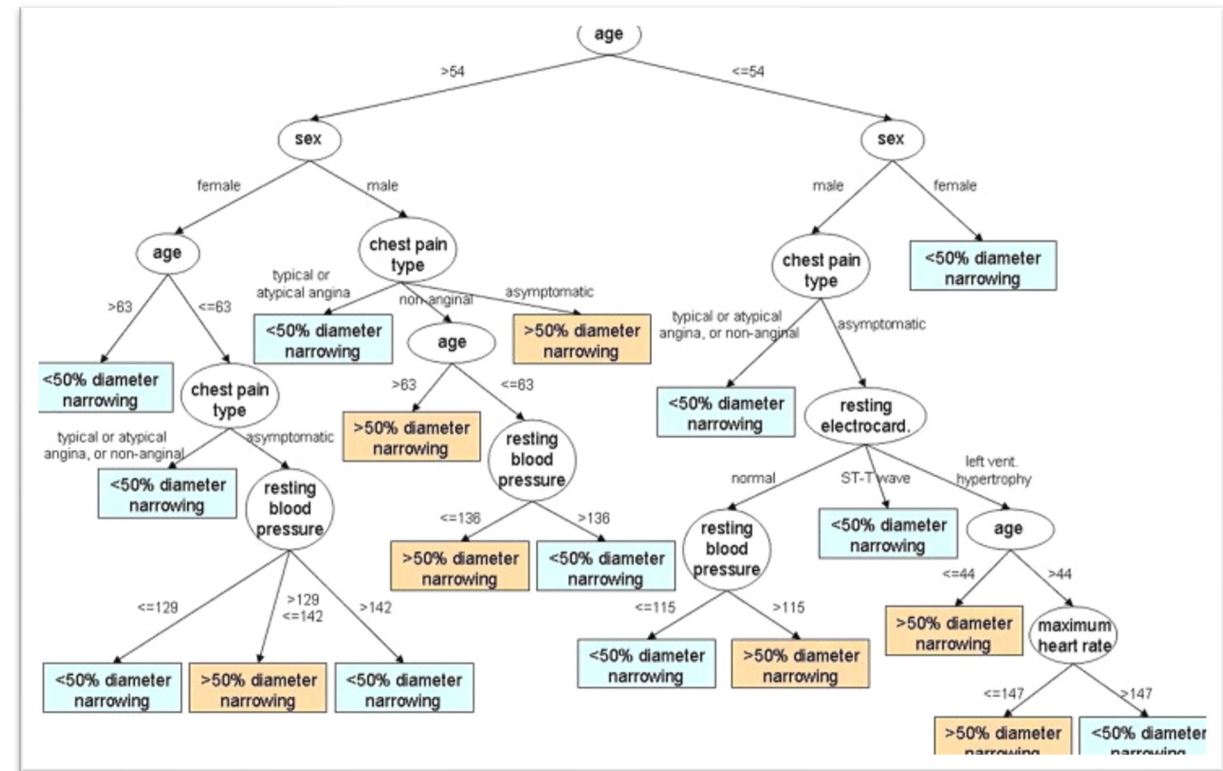
Why Use The Tree Data Structure?

In order to perform any operation on a linear data structure (such as arrays, linked lists, queues, etc.), the time complexity increases with the increase with the amount of data.

Given that trees are a non-linear data structure, they are often more computationally efficient to traverse, allowing for a quicker and easier way to access the data.

Trees are Ubiquitous in Computer Science

- Compilation of computer programs (abstract syntax trees)
- Natural language processing (NLP) (parse trees)
- File systems (folders, subfolders, ...)
- Biological systems (anatomy)
- Decision trees (e.g., medical diagnosis)
- Efficient searching and sorting



Decision Trees

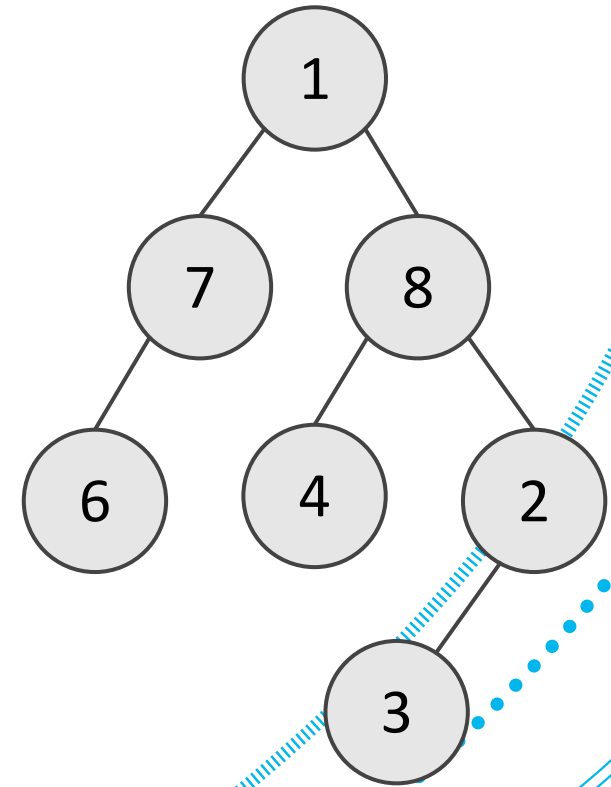


Decision Trees

Decision trees are a type of predictive modelling algorithm used in statistics, data mining, and ML.

They help in making decisions by learning simple decision rules inferred from the data features.

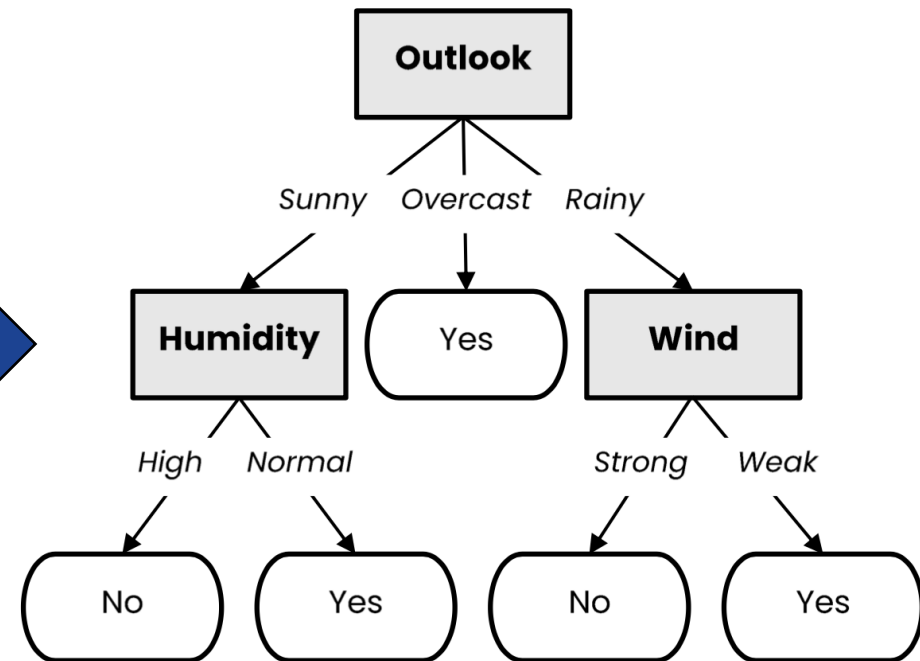
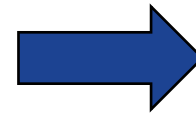
Decision trees are versatile, easy to understand, and applicable to a wide range of problems.



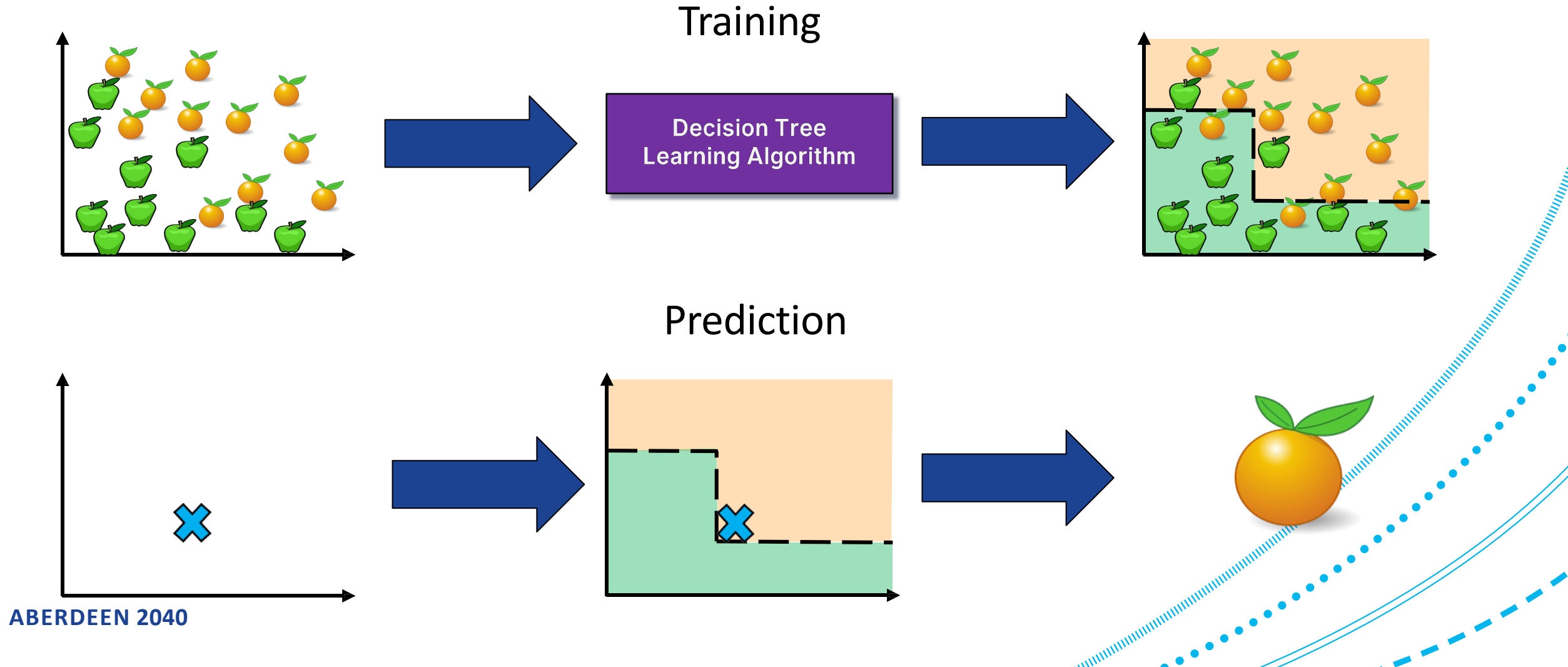
Decision Trees

We use labeled data to obtain a suitable decision tree for future predictions.

Outlook	Temperature	Humidity	Wind	Play Tennis?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rainy	Mild	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rainy	Mild	High	Strong	No

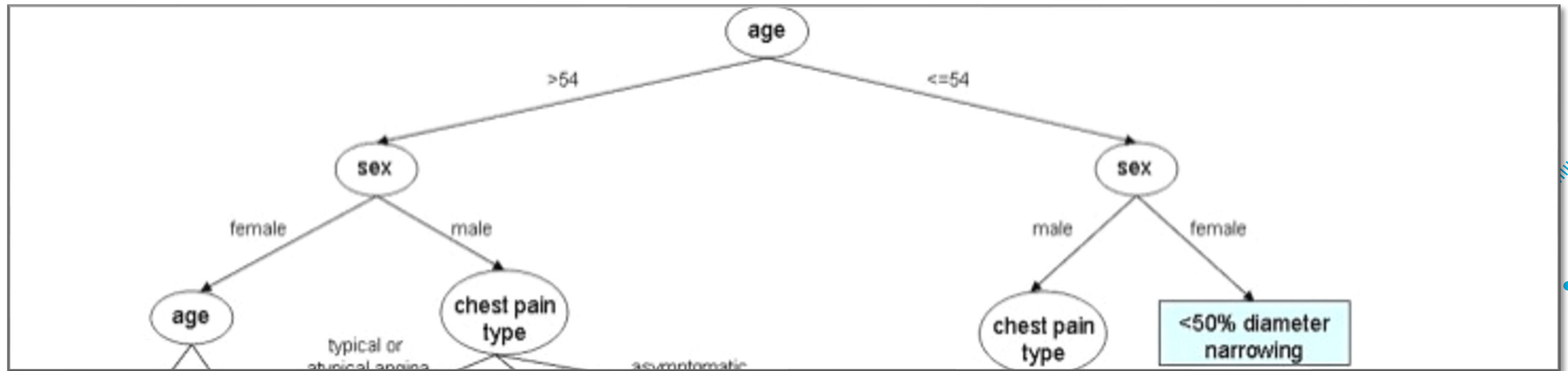


Decision Trees: Training and Prediction



Decision Trees: Interpretation

They are another example of a supervised learning approach with a high amount of interpretability.



Decision Trees: How to Train

Let's say we have a dataset of when we should play tennis.

How do we turn that into a decision tree? How do we 'split' this data up?

First, we need to learn about the 'purity' of the information.

Outlook	Temperature	Humidity	Wind	Play Tennis?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rainy	Mild	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rainy	Mild	High	Strong	No

Entropy

ABERDEEN 2040



Entropy

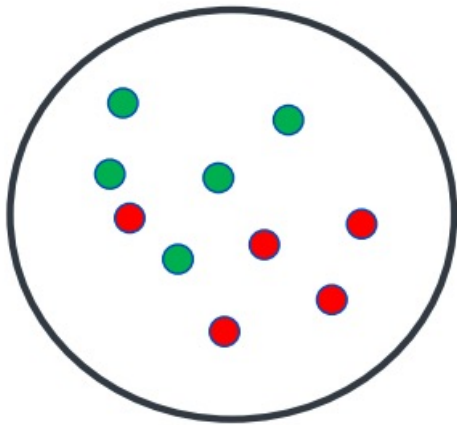
Entropy is the machine learning metric that measures the **impurity**, or *unpredictability*, in the system.

It is the measurement of disorder in the information being processed.

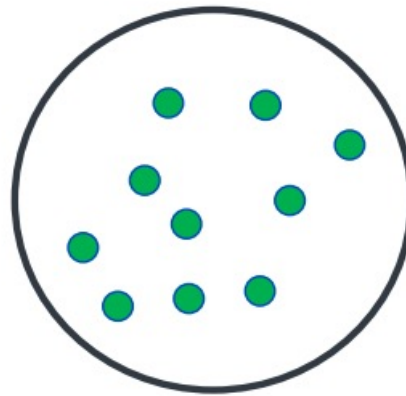
We use this measurement to determine how our decision tree should split the data.

Entropy

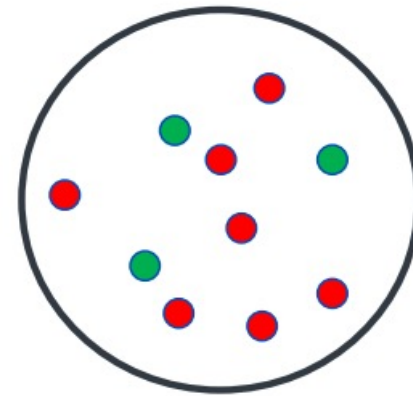
Different cases



Entropy=1



Entropy=0



Entropy=0.88

If a dataset contains an equal # of positive and negative data points, entropy is 1.

If a dataset contains only positive or only negative data points, then entropy is 0.

Entropy

Entropy measures the degree of randomness in data.

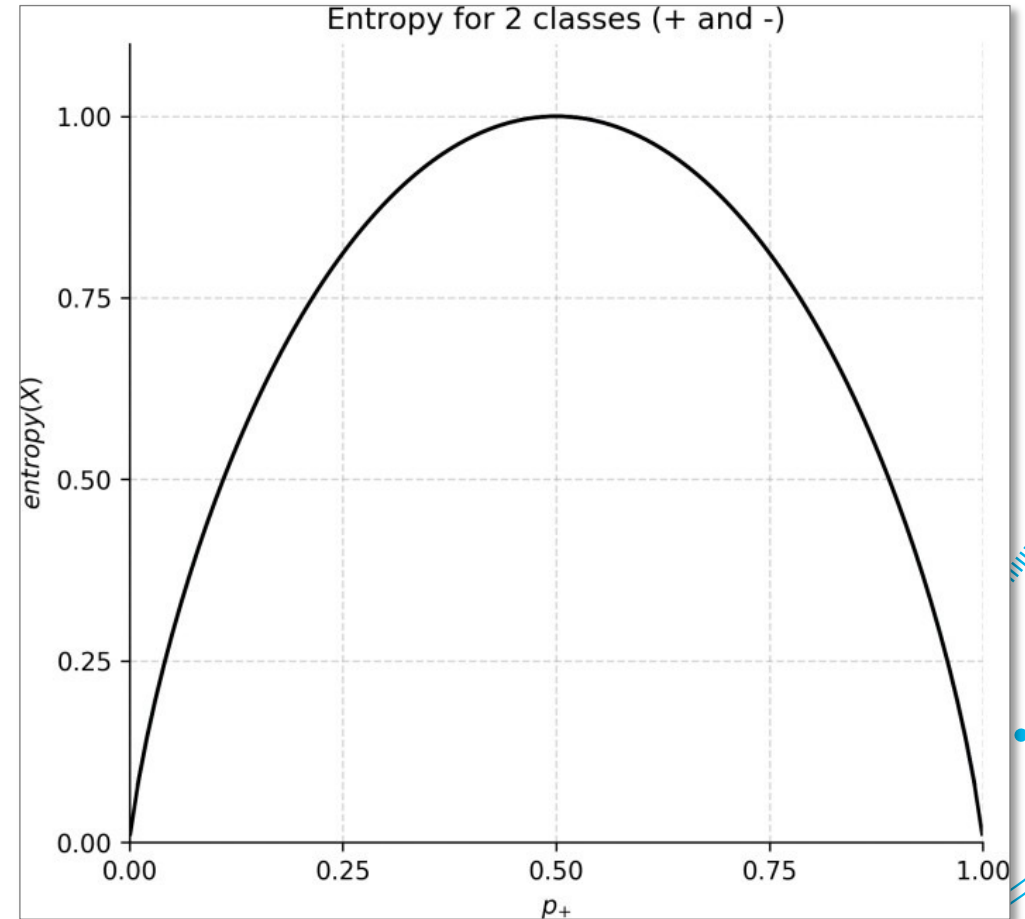
Low entropy



High entropy



Lower entropy implies greater predictability!



Calculating Entropy

For a set of samples X with k classes:

$$\text{entropy}(X) = - \sum_{i=1}^k p_i \log_2(p_i)$$

...where p_i is the proportion of elements of class i .

Calculating Entropy

In general, the entropy of a random variable V with values $\mathbf{v}_k$, each with probability $P(\mathbf{v}_k)$, is defined as Entropy:

$$H(V) = - \sum_k P(\mathbf{v}_k) \log_2 P(\mathbf{v}_k)$$

The entropy of a fair coin flip:

$$H(\text{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$$

Entropy

A random variable with only one value (e.g. a coin that always comes up heads), has no uncertainty.

Thus its entropy is defined as zero; We gain no information by observing its value.

$$- (1 * \log_2 1 + 0 * \log_2 0) = 0$$

(heads) (tails)

Calculating Entropy

$$H(V) = - \sum_k P(v_k) \log_2 P(v_k)$$

Example: If we had a total 10 data points in our dataset with 3 belonging to positive class and 7 belonging to negative class:

$$- (3/10 * \log_2 (3/10) + 7/10 * \log_2 (7/10)) \approx 0.876$$

High entropy means a low level of purity.

Information Gain



Information Gain

Entropy is a useful measure of disorder.

However, we still need a measure of “good” and “bad” for attributes. One way to do is to compute the information gain.

Information gain is defined as the pattern observed in the dataset, and the reduction in the entropy.

Information Gain

Mathematically, information gain can be expressed with this formula:

$$\text{Information Gain} = (\text{Entropy of parent node}) - (\text{Entropy of child node})$$

But essentially, we're looking at the entropy of a node after we split on an attribute, compared to before we split.

In other words, how much **information** do we **gain** by splitting?

Building a Decision Tree Using Information Gain

At every step in our decision tree, we want to maximise the information gained.

- An attribute with the highest information gain from a set should be selected as the parent (root) node.
- We then build child nodes for every value of that attribute.
- And then repeat iteratively until we construct the whole tree.

Choosing the Best Attribute

Example: Let's say we are looking to build a decision tree for whether we should wait for a table at a restaurant.

Example	Input Attributes										Output
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0-10</i>	$y_1 = \text{Yes}$
x_2	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30-60</i>	$y_2 = \text{No}$
x_3	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0-10</i>	$y_3 = \text{Yes}$
x_4	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10-30</i>	$y_4 = \text{Yes}$
x_5	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	$y_5 = \text{No}$
x_6	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0-10</i>	$y_6 = \text{Yes}$
x_7	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0-10</i>	$y_7 = \text{No}$
x_8	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0-10</i>	$y_8 = \text{Yes}$
x_9	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	$y_9 = \text{No}$
x_{10}	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10-30</i>	$y_{10} = \text{No}$
x_{11}	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0-10</i>	$y_{11} = \text{No}$
x_{12}	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30-60</i>	$y_{12} = \text{Yes}$

WillWait represents a binary variable, **our DV**, indicating whether or not we should wait.

Choosing the Best Attribute

Example: Let's say we are looking to build a decision tree for whether we should wait for a table at a restaurant.

Example	Input Attributes										Output
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x ₁	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	<i>y</i> ₁ = <i>Yes</i>
x ₂	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	<i>y</i> ₂ = <i>No</i>
x ₃	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	<i>y</i> ₃ = <i>Yes</i>
x ₄	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	<i>y</i> ₄ = <i>Yes</i>
x ₅	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	<i>y</i> ₅ = <i>No</i>
x ₆	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	<i>y</i> ₆ = <i>Yes</i>
x ₇	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	<i>y</i> ₇ = <i>No</i>
x ₈	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	<i>y</i> ₈ = <i>Yes</i>
x ₉	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	<i>y</i> ₉ = <i>No</i>
x ₁₀	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	<i>y</i> ₁₀ = <i>No</i>
x ₁₁	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	<i>y</i> ₁₁ = <i>No</i>
x ₁₂	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	<i>y</i> ₁₂ = <i>Yes</i>

Pat represents the number of Patrons; i.e. how many people are in the restaurant.

The possible values are:
None, Some, and Full.

Choosing the Best Attribute

At the root node of the restaurant problem, there are:

- 6 **Yes** cases
- 6 **No** cases

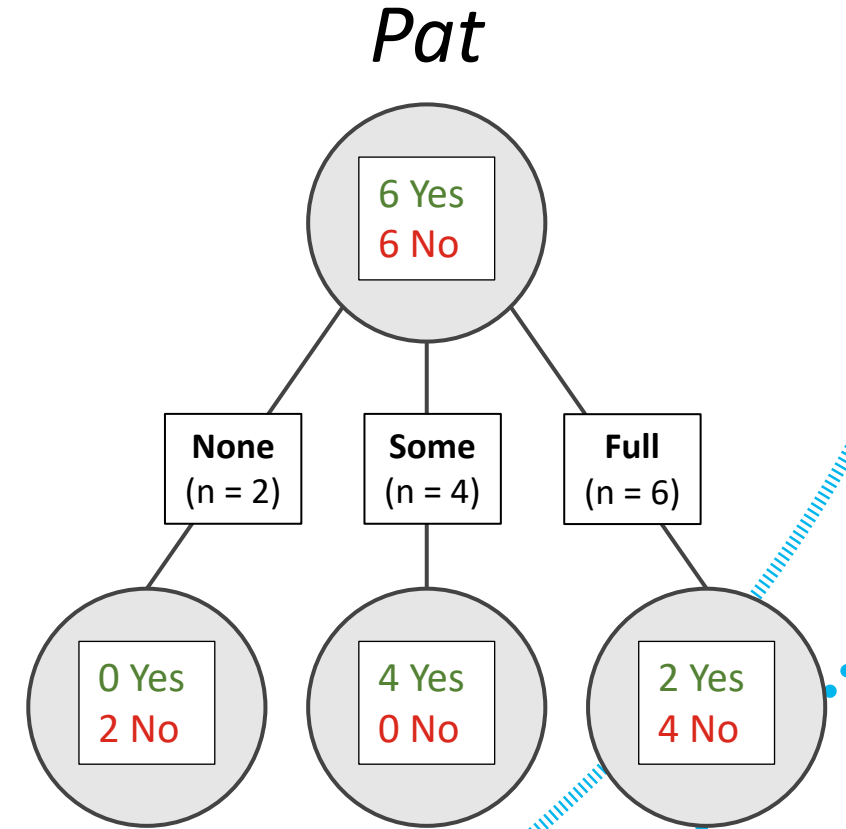
So we know that the Entropy(Parent), i.e. before we split, is 1.

Output
<i>WillWait</i>
$y_1 = \text{Yes}$
$y_2 = \text{No}$
$y_3 = \text{Yes}$
$y_4 = \text{Yes}$
$y_5 = \text{No}$
$y_6 = \text{Yes}$
$y_7 = \text{No}$
$y_8 = \text{Yes}$
$y_9 = \text{No}$
$y_{10} = \text{No}$
$y_{11} = \text{No}$
$y_{12} = \text{Yes}$

Choosing the Best Attribute

So what happens if we split on the *Patron* attribute?

We start off by calculating the Entropy for each value of Patron.

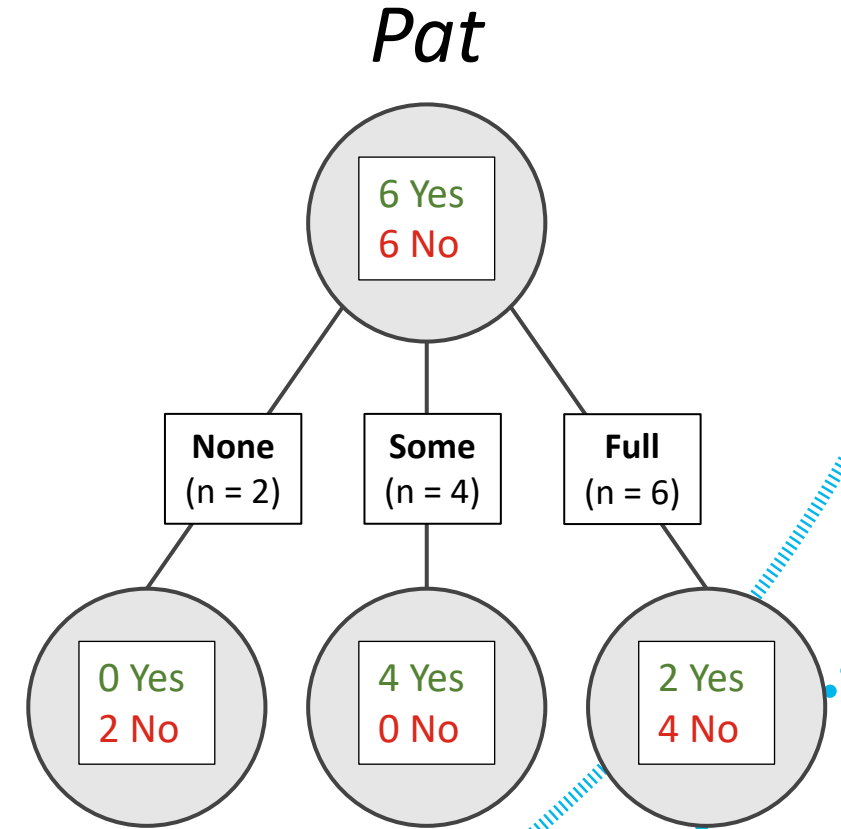


Choosing the Best Attribute

$$E(Pat = \text{None}) = 0$$

$$E(Pat = \text{Some}) = 0$$

$$\begin{aligned} E(Pat = \text{Full}) &= - (2/6 * \log_2 (2/6) + 4/6 * \log_2 (4/6)) \\ &\approx - (0.33 * -1.59 + 0.67 * -0.59) \\ &\approx - (-0.52 + -0.40) \\ &\approx 0.92 \end{aligned}$$



Choosing the Best Attribute

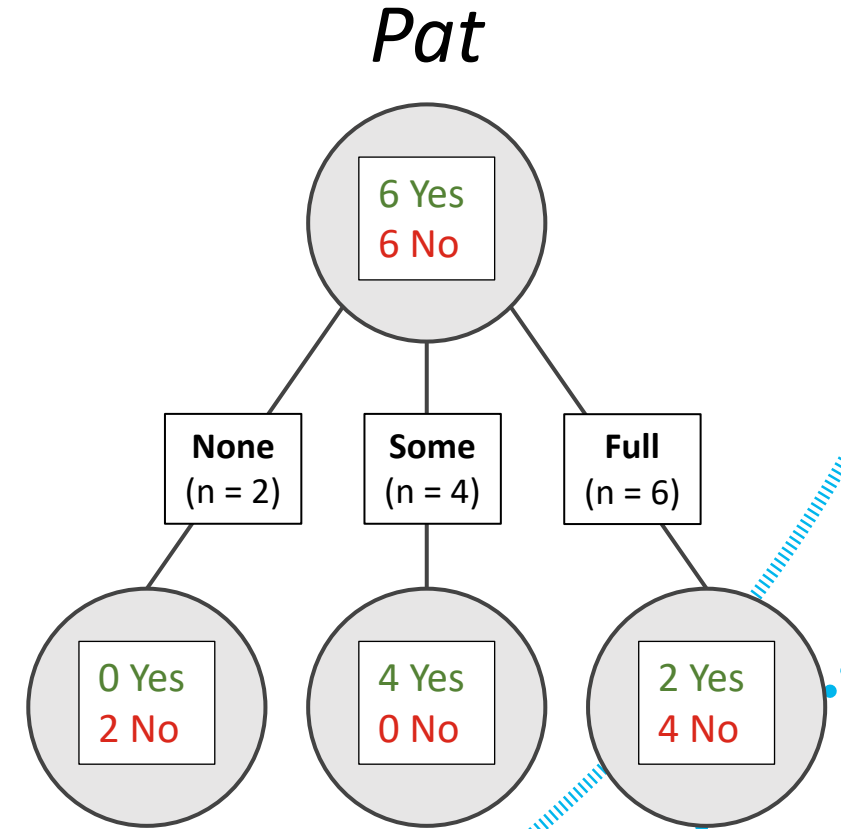
Next, we calculate the weighted average of the Entropy for each node.

$$E(Pat = \text{None}) = 0$$

$$E(Pat = \text{Some}) = 0$$

$$E(Pat = \text{Full}) \approx 0.92$$

$$\begin{aligned} E(Pat) &\approx (2/12 * 0) + (4/12 * 0) + (6/12 * 0.92) \\ &\approx 0.46 \end{aligned}$$

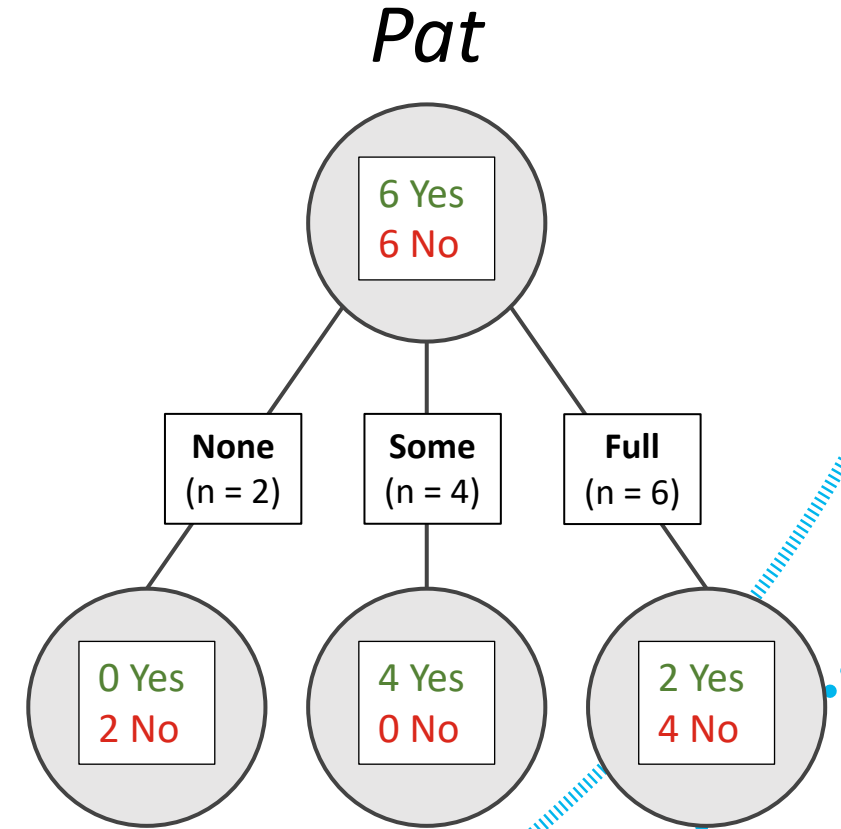


Choosing the Best Attribute

Finally, we calculate the Information Gain.

$$\text{IG} = (\text{Entropy of parent node}) - (\text{Entropy of child node})$$
$$\approx 1 - 0.46$$

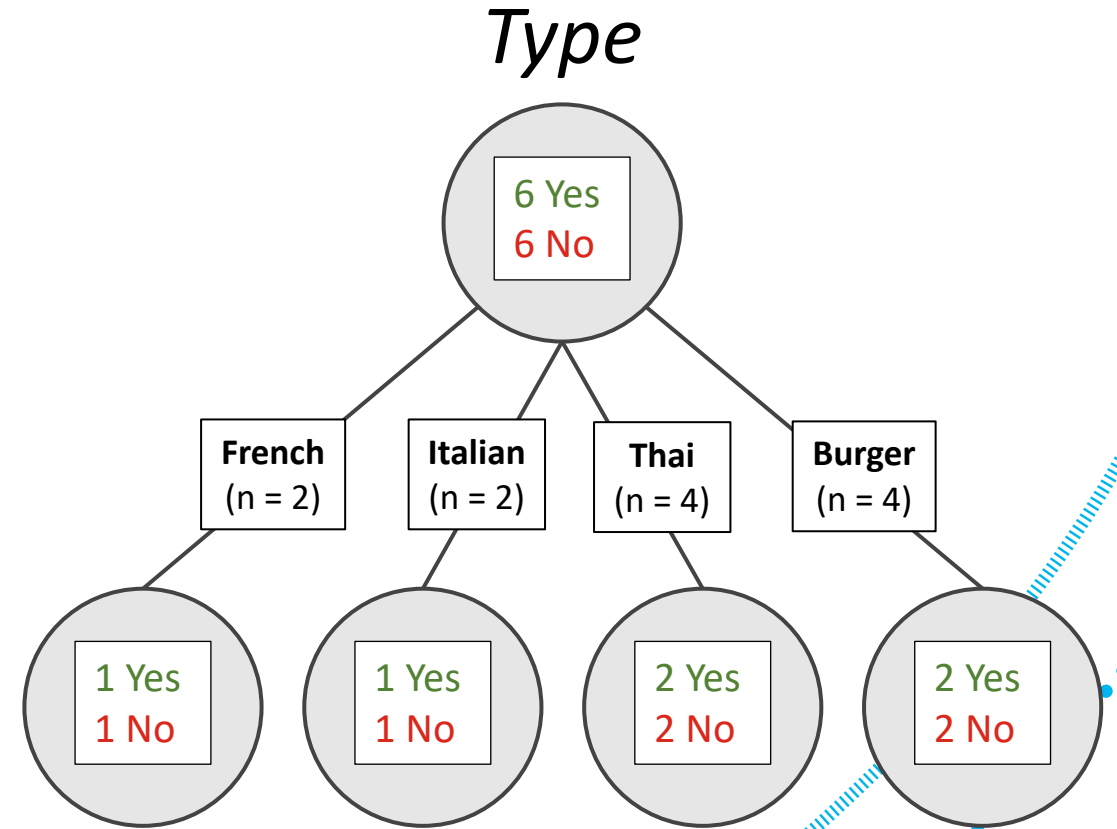
$$\text{IG}(\text{Pat}) \approx 0.54$$



Choosing the Best Attribute

Let's say we repeat that process with all other attributes (but we'll just show one as an example):

The *Type* of restaurant.



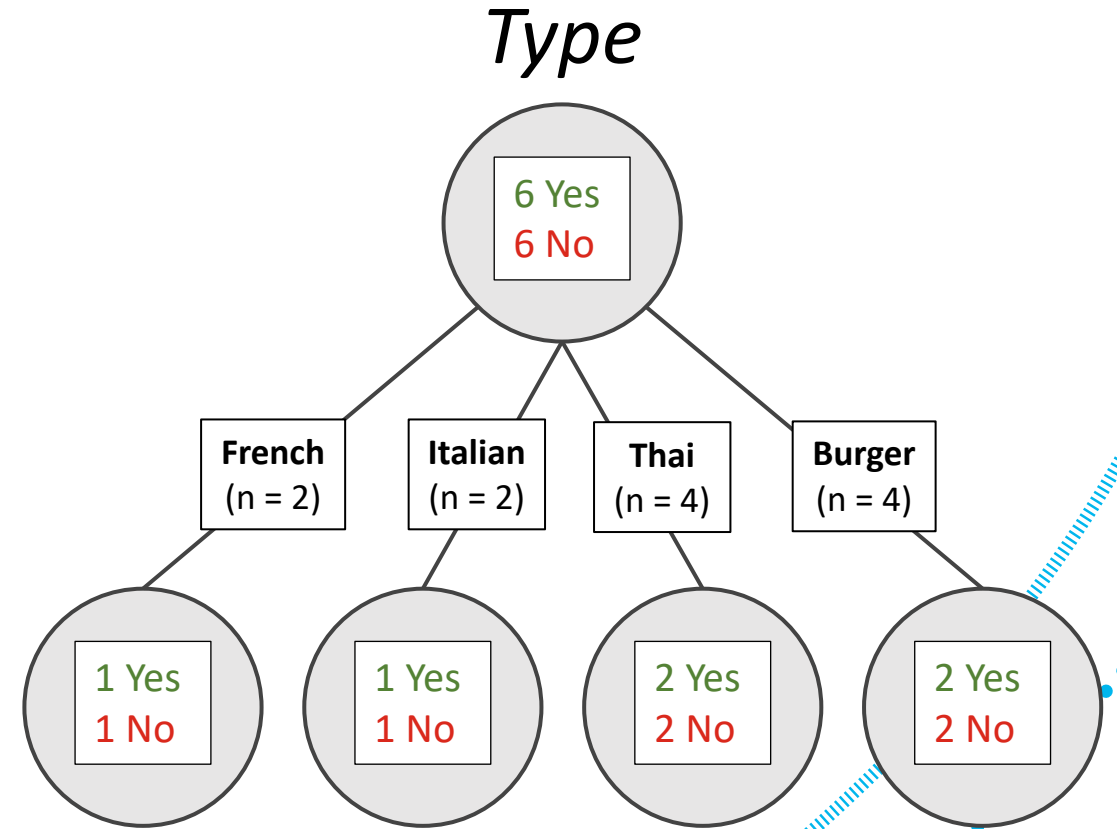
Choosing the Best Attribute

$$E(\textit{Type} = \textit{French}) = 1$$

$$E(\textit{Type} = \textit{Italian}) = 1$$

$$E(\textit{Type} = \textit{Thai}) = 1$$

$$E(\textit{Type} = \textit{Burger}) = 1$$



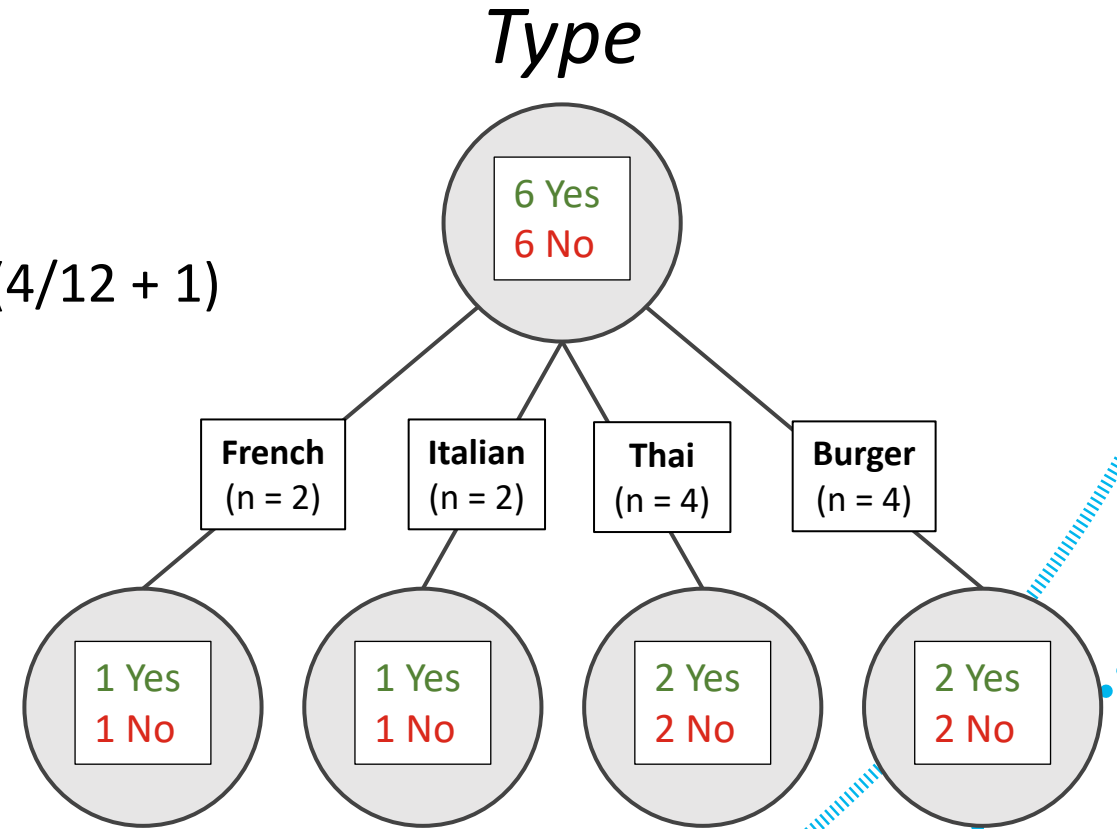
Choosing the Best Attribute

Weighted Entropy:

$$E(Pat) = (2/12 * 1) + (2/12 * 1) + (4/12 * 1) + (4/12 * 1) \\ = 1$$

Information Gain:

$$IG(Type) = 1 - 1 \\ = 0$$



Choosing the Best Attribute

In other words, we gain more information by splitting on *Pat* than by *Type*.

